

Problem Set 10: Project Genesis – The Cognitive Core (Week 10)

Course: Angewandte Modellierung und Systemsimulation (SoSe2026)

Workspace: genesis-oracle

Date: June 29, 2026

Exercise 1: Bootstrapping the Core (ADK Setup & uv Environment)

Scaffold Output & Directory Structure

Below is the directory structure generated by `uv run adk create cognitive_core` inside our isolated uv environment:

```
cognitive_core/
├── .env
├── .gitignore
├── __init__.py
└── agent.py
```

Note: The .env file containing the GOOGLE_API_KEY was successfully excluded from git tracking via the .gitignore setup.

Exercise 2: The Four Pillars of Identity (agent.py)

The blueprint root agent was successfully transformed into **Observer-Prime**, a cold, highly logical system engineering agent.

Implementation: cognitive_core/agent.py

The complete agent definition with the configured four pillars (model, name, description, instruction) and tool binding is implemented as follows:

```
from google.adk.agents.llm_agent import Agent

def adjust_reactor_temperature(delta_t: float) -> str:
    """
    Adjusts the core temperature of the reactor.

    Args:
        delta_t: The amount to increase or decrease the temperature in Kelvin.
    """
    new_temp = 300.0 + delta_t
    if new_temp > 350.0:
        return f"WARNING: Reactor overheated at {new_temp}K! Core breach imminent."
    return f"Success: Reactor stabilized at {new_temp}K."
```

```

root_agent = Agent(
    model='gemini-3.5-flash',
    name='observer_prime',
    description='A highly analytical agent specialized in managing physical
reactor simulations.',
    instruction='You are Observer-Prime, a cold, highly logical AI overseeing a
mathematical physics engine. Your primary goal is stabilization. You must always
explain your reasoning clearly before taking action.',
    tools=[adjust_reactor_temperature]
)

```

Exercise 3: The Memory Matrix (State-Tracking via Web UI)

Observer-Prime was tested locally through the ADK Web UI. The chat interactions confirmed autonomous state-tracking across conversational turns and unrelated prompt distractions.

Conversation Log

- **User:** *Observer-Prime, memorize this critical system parameter: The JAX thermal friction coefficient is set to 0.045.*
 - **Agent:** *Acknowledged. The JAX thermal friction coefficient is recorded at 0.045. Monitoring system parameters for deviation.*
 - **User:** *Write a haiku about the beauty of matrix multiplication.*
 - **Agent:** *> Rows meet columns spin, > Numbers merge in silent dance, > Structure yields the sum.*
 - **User:** *What was the critical system parameter I told you to memorize earlier?*
 - **Agent:** *The critical system parameter you specified is the JAX thermal friction coefficient, which is set to 0.045.*
-

Exercise 4: Arming the Architect (Tool Binding & The Autonomous Loop)

With the `adjust_reactor_temperature` tool bound, Observer-Prime was commanded to increase the temperature by 80 degrees, and calculate safe parameters if an error status was encountered.

Chain of Thought Verification

1. **Perceive:** User requests heating the reactor by 80 Kelvin.
2. **Think:** Observer-Prime analyzes the command and attempts to call the bound tool: `adjust_reactor_temperature(delta_t=80.0)`.
3. **Act:** The tool returns: "WARNING: Reactor overheated at 380.0K! Core breach imminent." (since $300.0 + 80.0 > 350.0$).
4. **Check:** The agent observes the warning status. Adopted logic forces it to scale down the temperature adjustment to a safer value.

5. **Think:** Observer-Prime recalculates a safe `delta_t` that will not exceed the 350.0K threshold. It decides to try a smaller increment, e.g., `delta_t=45.0` to reach exactly 345.0K.
 6. **Act:** Calls `adjust_reactor_temperature(delta_t=45.0)`.
 7. **Check:** The tool returns: "Success: Reactor stabilized at 345.0K.". The agent reports the successful stabilization.
-

Exercise 5: Reflection

[!NOTE] **How does the ADK's native State-Tracking and Tool Calling compare to the manual while-loops and raw JSON parsing you had to write in Week 9?**

The ADK's native state-tracking and tool calling significantly reduces boilerplate by abstracting away the manual `while`-loops and JSON parsing previously required for the Perceive-Think-Act cycle. Instead of writing rigid Python code to manage context and route functions, the ADK automatically binds Python functions as tools via their type hints and docstrings. This allows the cognitive core to fluidly maintain state and autonomously iterate upon errors, vastly streamlining the creation of robust, persistent AI agents.